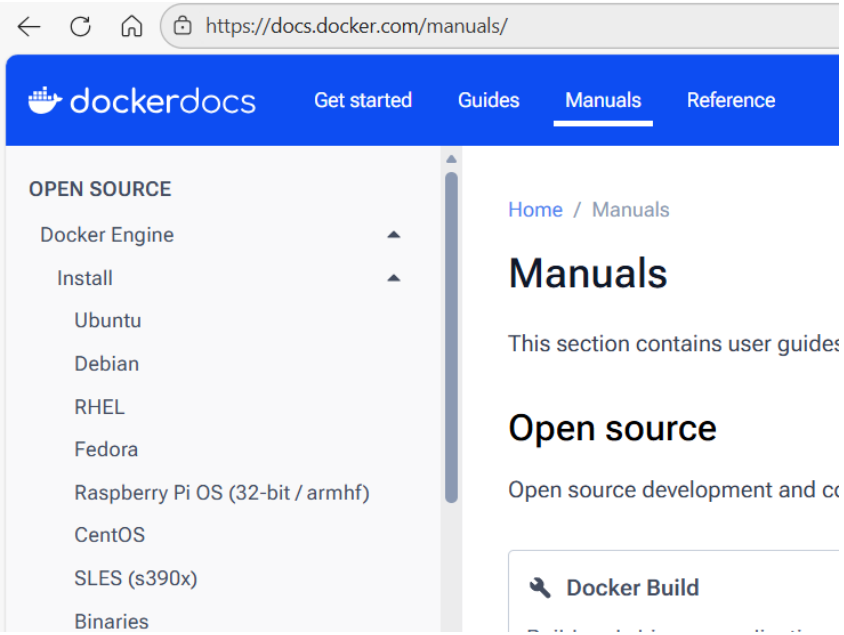
Docker

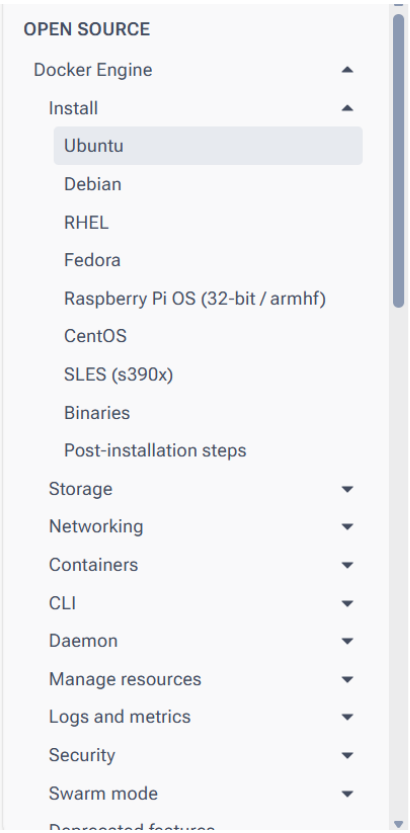
Parte I

PAULO R T CÂNDIDO

Instalação

Seguir orientação no site oficial

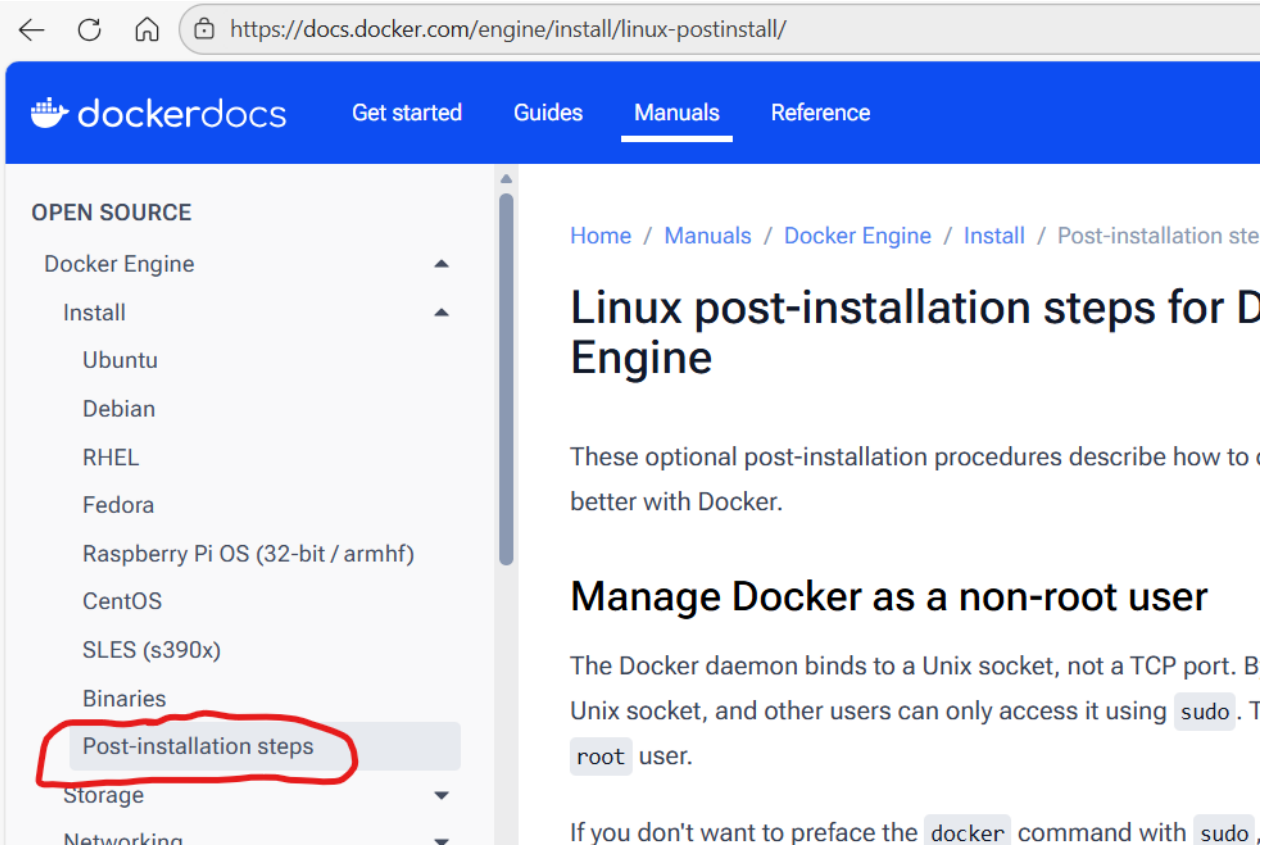




https://docs.docker.com/manuals/

Instalação

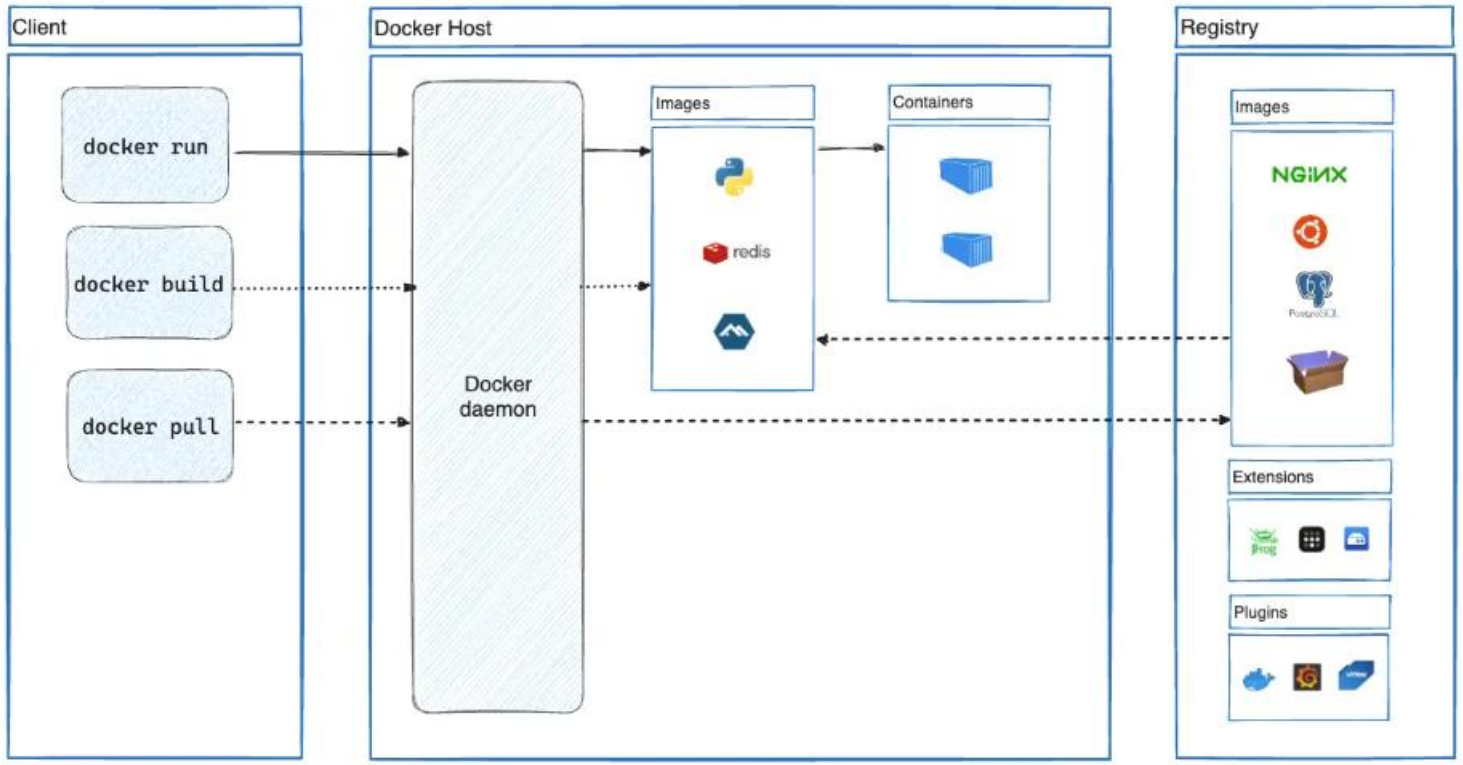
Seguir orientação no site oficial



https://docs.docker.com/engine/install/linux-postinstall/

Conceitos

O daemon Docker atende a requisições feitas na API Docker e gerencia o Docker objetos: imagens, contêineres, redes e volumes.



O que é Docker? | Docker Docs

Cliente Docker (): forma como usuários interagem com Docker. Utilizam API para se comunicar com daemon.

Registro: armazena imagens Docker. O Docker Hub é um centro público padrão. É possível administrar registros particulares.

Imagem é um modelo somente leitura com instruções (arquivos e configurações) para criar um Docker contêiner

Container é uma instância executável de uma imagem

Docker Compose: usado para definir multiplos containers e suas configurações em um único arquivo YAML.

Introdução aos comandos Docker

```
mkdir testedocker && cd testedocker

docker run -d --name=meucontainer node:24-alpine node -v
```

Instancia container a partir da imagem (modelo) “node:24-alpine” (Ubuntu + NodeJS)

Faz download automaticamente do Docker Hub caso a imagem não exista localmente.

“-d” implica que o container será desconectado do terminal, roda em background.

“--name" é o nome do container

“node -v” é o que será executado (print a versão do NodeJS no terminal do container)

Introdução aos comandos Docker

Visualiza imagens baixadas

```
prtcandido@PauloAcer:~/testedocker$ docker images

IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA
node:24-alpine     4f696fbf39f3       224MB         56.5MB      U
prtcandido@PauloAcer:~/testedocker$
```

Visualiza container ativos (em execução)

```
prtcandido@PauloAcer:~/testedocker$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS   PORTS   NAMES
prtcandido@PauloAcer:~/testedocker$
```

Visualiza todos os container, mesmo os que encerraram

```
prtcandido@PauloAcer:~/testedocker$ docker ps -a
CONTAINER ID   IMAGE            COMMAND                  CREATED          STATUS                      PORTS     NAMES
94a12ac7f5d4   node:24-alpine   "docker-entrypoint.s…"   13 minutes ago   Exited (0) 13 minutes ago             meucontainer
prtcandido@PauloAcer:~/testedocker$
```

Introdução aos comandos Docker

Ativar novamente o container e consultar o log (terminal do container)

```
prtcandido@PauloAcer:~/testedocker$ docker start meucontainer
meucontainer
prtcandido@PauloAcer:~/testedocker$ docker logs meucontainer
v24.13.1
v24.13.1
```

Remover o container

```
prtcandido@PauloAcer:~/testedocker$ docker ps -a
CONTAINER ID   IMAGE            COMMAND                  CREATED
94a12ac7f5d4   node:24-alpine   "docker-entrypoint.s…"   56 minutes
prtcandido@PauloAcer:~/testedocker$ docker rm 94a12ac7f5d4
94a12ac7f5d4
prtcandido@PauloAcer:~/testedocker$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NA
prtcandido@PauloAcer:~/testedocker$
```

Introdução aos comandos Docker

Remover a imagem localmente

```
prtcandido@PauloAcer:~/testedocker$ docker images

IMAGE             ID             DISK USAGE   CONTENT SIZE   EXTRA
node:24-alpine    4f696fbf39f3        224MB         56.5MB
prtcandido@PauloAcer:~/testedocker$ docker rmi 4f696fbf39f3
Untagged: node:24-alpine
Deleted: sha256:4f696fbf39f383c1e486030ba6b289a5d9af541642fc78ab197e584a113b9c03
prtcandido@PauloAcer:~/testedocker$ docker images

IMAGE    ID             DISK USAGE   CONTENT SIZE   EXTRA
prtcandido@PauloAcer:~/testedocker$
```

Imagem com programa Javascript

Criar programa Javascript na máquina local

```
prtcandido@PauloAcer:~/testedocker$ vi teste.js
prtcandido@PauloAcer:~/testedocker$ cat teste.js
function soma(a,b) {
  return a + b
}
console.log( soma(2,3)  )
prtcandido@PauloAcer:~/testedocker$
```

Dockerfile – define imagem incluindo programa javascript

```
# syntax=docker/dockerfile:1

# Usa imagem Linux com Node.js de pequeno tamanho
FROM node:24-alpine

# Configura o diretório de trabalho dentro do container para /app
WORKDIR /app

# Copia o arquivo javascript (diretório atual máquina física) para o diretório de trabalho
no container
COPY ./teste.js .

# Configura o comando padrão para executar o programa (node teste.js)
CMD ["node", "teste.js"]
```

Build da imagem

docker build -t testeimagemjs:1.0 .

-t defini a Tag da imagem nome:versão

Note o “ponto” na final do comando, refere-se ao diretório atual

Build da imagem

```
prtcandido@PauloAcer:~/testedocker$ vi dockerfile
prtcandido@PauloAcer:~/testedocker$ cat dockerfile
# syntax=docker/dockerfile:1

# Usa imagem Linux com Node.js de pequeno tamanho
FROM node:24-alpine

# Configura o diretório de trabalho dentro do container para /app
WORKDIR /app

# Copia o arquivo javascript (diretório atual máquina física) para o diretório de trabalho no container
COPY ./teste.js .

# Configura o comando padrão para executar o programa (node teste.js)
CMD ["node", “teste.js"]
prtcandido@PauloAcer:~/testedocker$ docker build -t testeImagemJS:1.0 .
[+] Building 0.0s (0/0)                                                                                    docker:default
ERROR: failed to build: invalid tag "testeImagemJS:1.0": repository name must be lowercase
prtcandido@PauloAcer:~/testedocker$ docker build -t testeimagemjs:1.0 .
[+] Building 8.9s (8/11)                                                                                   docker:default
 => [internal] load build definition from dockerfile                                                                0.0s
 => => transferring dockerfile: 466B                                                                                0.0s
 => resolve image config for docker-image://docker.io/docker/dockerfile:1                                           1.1s
 => [auth] docker/dockerfile:pull token for registry-1.docker.io                                                    0.0s
 => CACHED docker-image://docker.io/docker/dockerfile:1@sha256:b6afd42430b15f2d2a4c5a02b919e98a525b785b1aaff16747d2f623364e39b6   0.0s
 => => resolve docker.io/docker/dockerfile:1@sha256:b6afd42430b15f2d2a4c5a02b919e98a525b785b1aaff16747d2f623364e39b6   0.0s
 => [internal] load metadata for docker.io/library/node:24-alpine                                                   1.9s
 => [auth] library/node:pull token for registry-1.docker.io                                                         0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 2B                                                                                     0.0s
 => [1/3] FROM docker.io/library/node:24-alpine@sha256:4f696fbf39f383c1e486030ba6b289a5d9af541642fc78ab197e584a113b9c03   5.7s
 => => resolve docker.io/library/node:24-alpine@sha256:4f696fbf39f383c1e486030ba6b289a5d9af541642fc78ab197e584a113b9c03   0.0s
 => => sha256:2906ccb85a84d0354480fdd85480d69a98940935ea465513deb8afb42a396923 1.05MB / 1.26MB                      5.6s
 => => sha256:8ba8341791ce72c3cdec81ab7f3abf29803e34a45ed9a8389accf9dbc4fa8355 38.80MB / 51.06MB                    5.6s
 => => sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153 3.86MB / 3.86MB                      3.5s
 => => sha256:cf98c2603c481adea278ab1e7074811323852744 64b434be7dc87f3ca249c74b 445B / 445B                          0.8s
 => => extracting sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153                           0.1s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 92B                                                                                    0.0s
```

Instanciar container

```
prtcandido@PauloAcer:~/testedocker$ docker images

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
testeimagemjs:1.0    c8d866f754c1        224MB         56.2MB
prtcandido@PauloAcer:~/testedocker$ docker run -d --name meujs testeimagemjs:1.0
5dbb0fa55a7f1d14c11a1d31e12839358aa6cf4959db31e9a7e3288d04b4d595
prtcandido@PauloAcer:~/testedocker$ docker logs meujs
5
prtcandido@PauloAcer:~/testedocker$ |
```

Exemplo 2 – Aplicação Web

Obter fontes da aplicação

```
git clone https://github.com/docker/getting-started-app.git
```

Entrar na pasta do projeto

```
cd getting-started-app
```

Criar o dockerfile

```
# syntax=docker/dockerfile:1
FROM node:24-alpine
WORKDIR /app
COPY . .
RUN npm install --omit=dev
CMD ["node", "src/index.js"]
EXPOSE 3000
```

Instala as dependências necessárias para ambiente de produção. Omite dependências usadas para desenvolvimento.

Executa o comando: node src/index.js

Porta do container associada a aplicação

Criar imagem

```
docker build -t nodeexemplo2:1.0 .
```

Instanciar container

Instanciar container

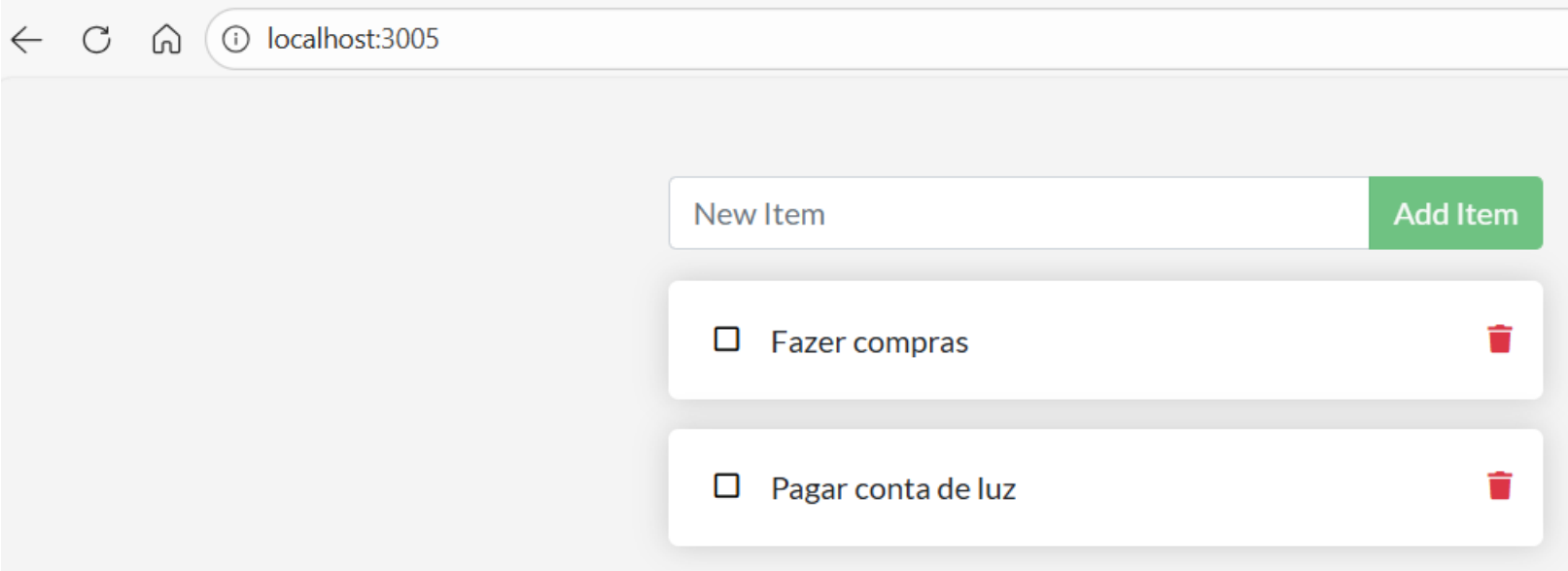
docker run -d --name instancia1 -p 127.0.0.1:3005:3000 nodeexemplo2:1.0

-d executa em segundo plano

--name é o nome dado para a instância

-p é o IP (localhost), a porta no Host e a porta interna do container. Ao enviar requisições para a porta 3005 da máquina física, ela é repassada para a 3000 do conainer

Consulta aplicação via nagegador



Para, reiniciar e consultar o container

Para container
docker stop instancia1

Reiniciar container
docker start instancia1

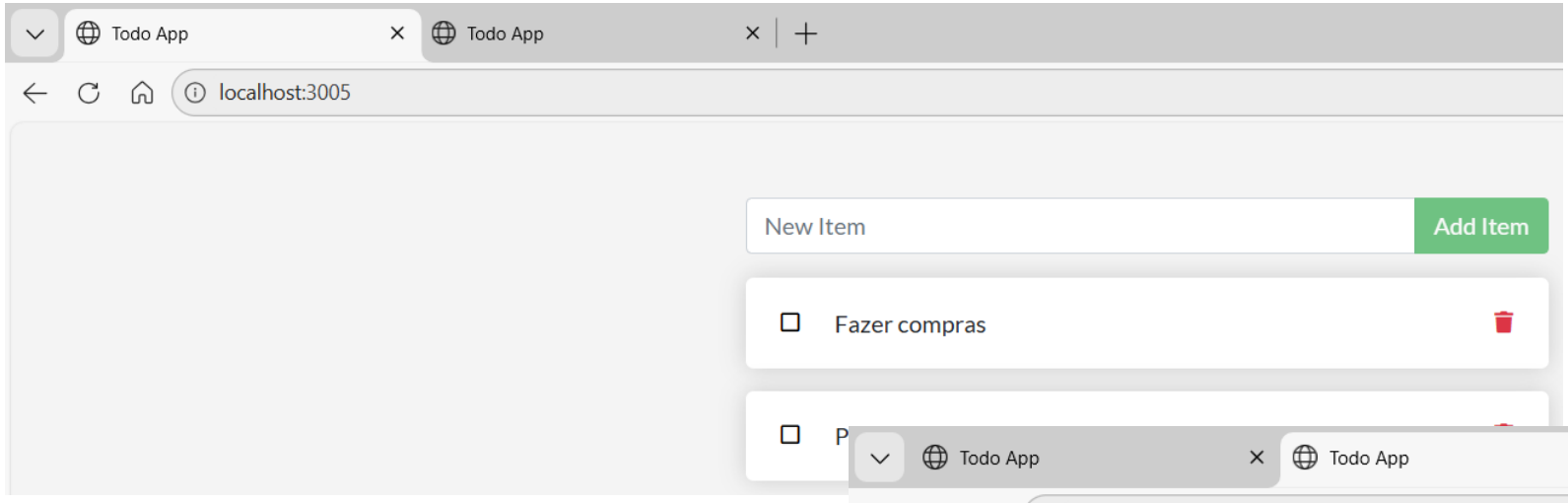
Entrar no container (imagem node:24-alpine não tem bash, usado sh)
docker exec -it instancia1 sh
-it para entrar no container de forma interativa (abre o terminal do container)
sh é o terminal (shell) a ser usado

Consultar logs do container
docker logs instancia1

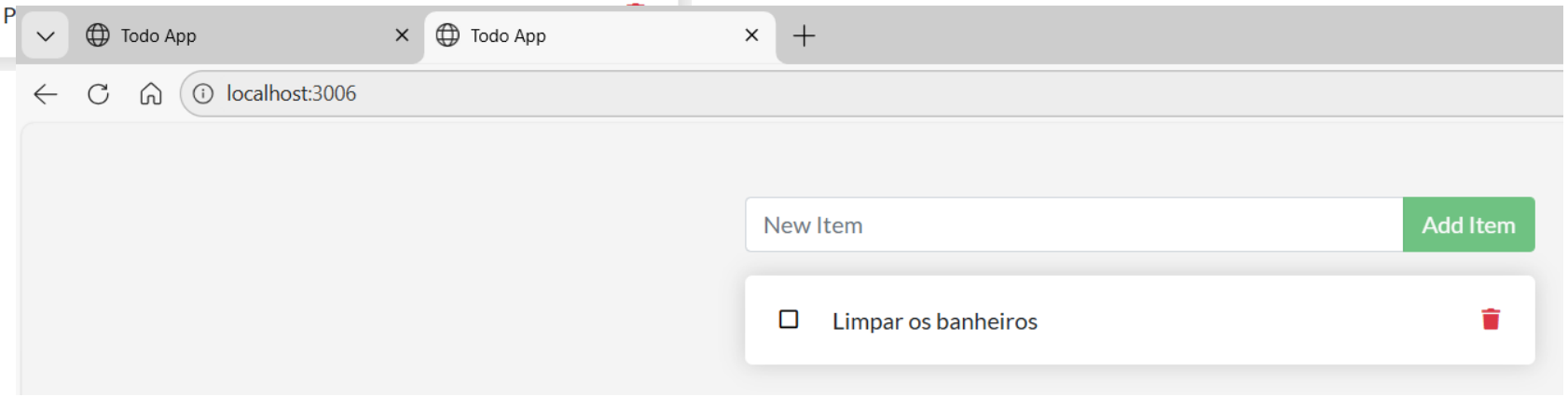
Criar segunda instância do container

docker run -d --name instancia2 -p 127.0.0.1:3006:3000 nodeexemplo2:1.0

```
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker run -d --name instancia2 -p 127.0.0.1:3006:3000 nodeexemplo2:1.0
b8115fc5683d9a3f308f77ba4eac0a2dbb7cbf1c1ef64627f627b7616f4ccc5f
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS          PORTS                      NAMES
b8115fc5683d   nodeexemplo2:1.0   "docker-entrypoint.s…"   10 seconds ago   Up 9 seconds    127.0.0.1:3006->3000/tcp   instancia2
2a78b743be1c   nodeexemplo2:1.0   "docker-entrypoint.s…"   16 minutes ago   Up 16 minutes   127.0.0.1:3005->3000/tcp   instancia1
```



Cada container mantem seu próprio sistema de arquivos, não compartilhando dados.



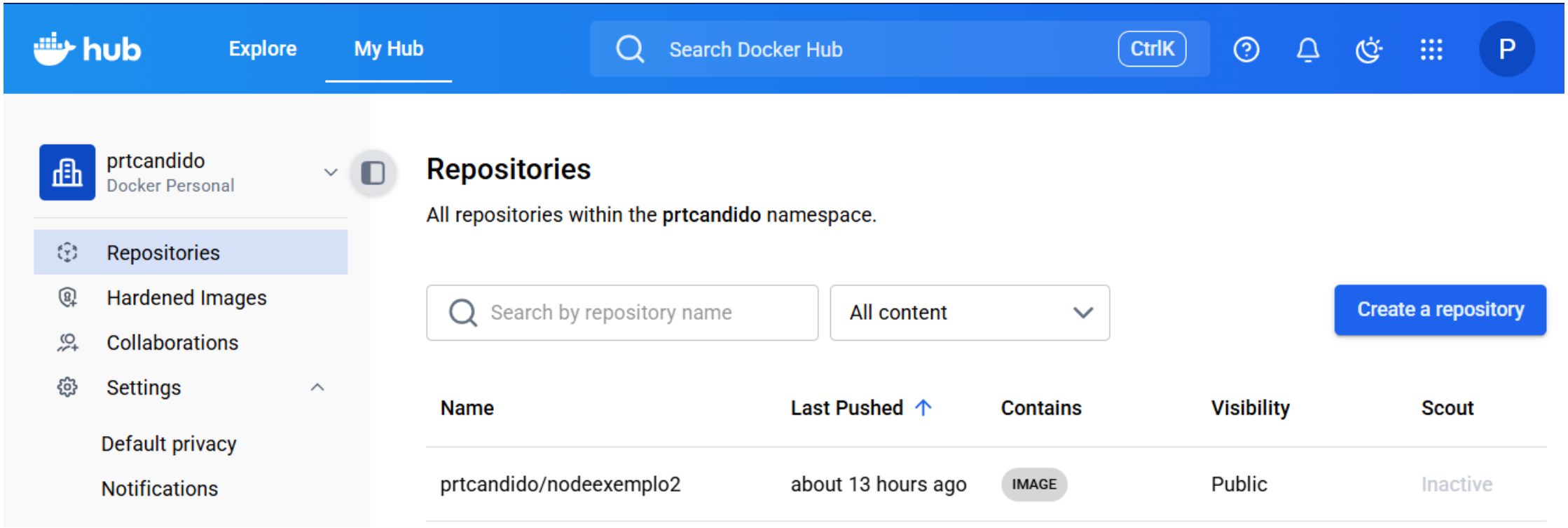
Se remover o container (Docker rm) os dados serão perdidos.

Alternativa: armazenar dados na máquina física (Host)

Compartilhar imagem no Docker Hub

Criar conta no Docker Hub
https://app.docker.com

Criar repositório (nodeexemplo2)

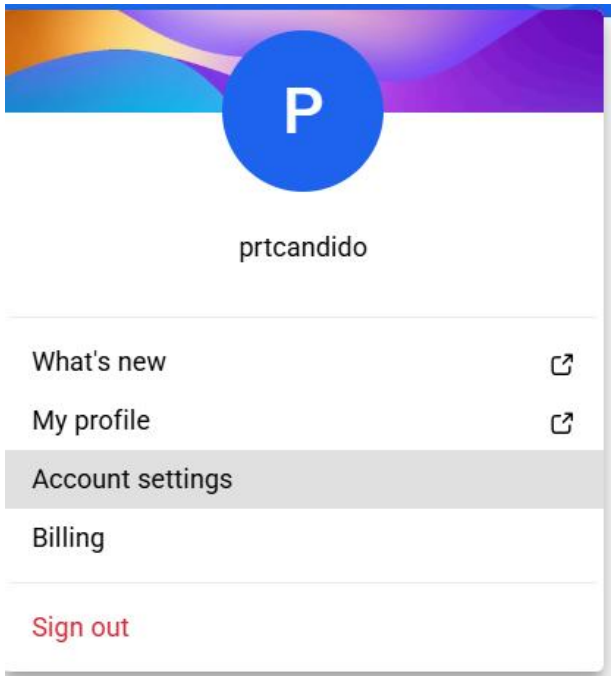


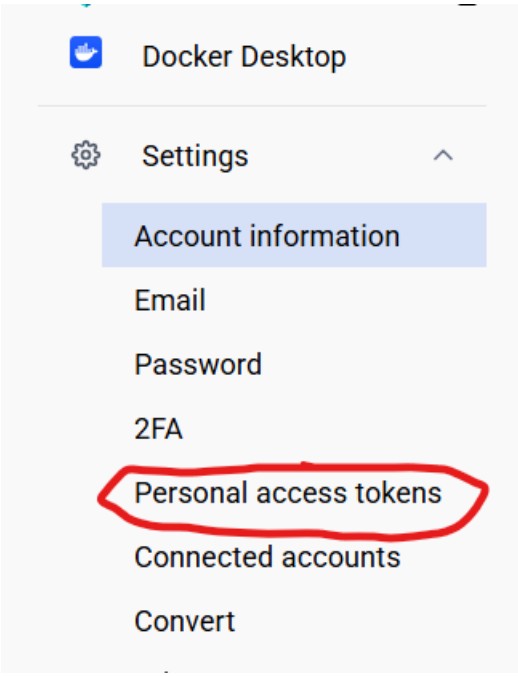
Compartilhar imagem no Docker Hub

Logar via terminal

docker login -u prtcandido

Será solicitado um token (imagens abaixo do Docker Hub)





Compartilhar imagem no Docker Hub

Gerar nova tag da imagem, inserindo o nome de sua conta de usuário

```
docker tag nodeexemplo2:1.0 prtcandido/nodeexemplo2:1.0
```

Enviar imagem para o Docker Hub

```
docker push prtcandido/nodeexemplo2:1.0
```

Remover todos os containers

```
docker rm -f $(docker ps -aq)
```

Remover todas as imagens

```
docker rmi $(docker images -q)
```

Criar container a partir do registro (Docker Hub)

```
docker run -d --name=inst1 -p 3001:3000 prtcandido/nodeexemplo2:1.0
```

Armazenamento de dados no Host

Para que os dados não sejam perdidos se o container for removido, podem ser armazenados na máquina física (Host) por meio de Volume.

```
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker volume create volumeexemplo
volumeexemplo
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker volume ls
DRIVER    VOLUME NAME
local     volumeexemplo
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker volume inspect volumeexemplo
[
    {
        "CreatedAt": "2026-02-26T11:20:29-03:00",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/volumeexemplo/_data",
        "Name": "volumeexemplo",
        "Options": null,
        "Scope": "local"
    }
]
prtcandido@PauloAcer:~/testedocker2/getting-started-app$
```

Os comandos acima criam um volume de armazenamento. Com o comando "docker volume inspect" é possível encontrar o local (caminho) na máquina física onde os arquivos do volume serão armazenados.

Instanciar container usando o volume

Recriar inst1 usando o volume

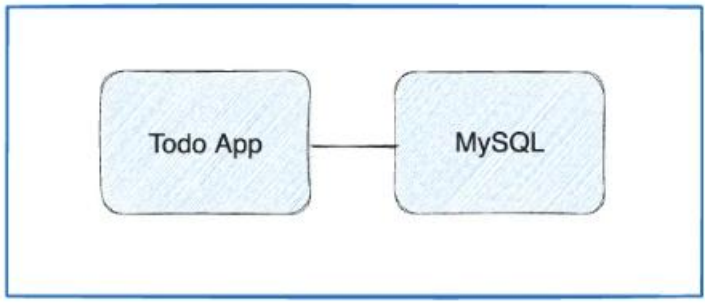
```
docker rm -f $(docker ps -aq)

docker run -d --name inst1 -p 127.0.0.1:3001:3000 --mount
type=volume,src=volumeexemplo,target=/etc/todos prtcandido/nodeexemplo2:1.0
```

Em --mount:
"type=volume" especifica o tipo de montagem (outra possibilidade é bind)
src é nome do volume
target é o caminho no container onde a aplicação acessa os dados (onde está o arquivo do SQL Lite). Este caminho é redirecionado para o volume.

Exercício: remova o container e o recrie. Os dados não mais serão perdidos pois estão no volume na máquina física.

Multiplos containers em rede



Criar rede

docker network create todo-app

Cria container com base na imagem mysql:8:0

```
docker run -d \
   --name meubd \
   --network todo-app --network-alias mysql \
   -v todo-mysql-data:/var/lib/mysql \
   -e MYSQL_ROOT_PASSWORD=secret \
   -e MYSQL_DATABASE=todos \
   mysql:8.0
```

Usado como referência ao container onde está o mysql

Volume onde os arquivos do banco serão criados.

Senha do usuário Root

Banco de dados

Container da aplicação

Obs.: entrar na pasta do projeto (getting-started-app)

```
docker run -dp 127.0.0.1:3000:3000 \
  -w /app \
  -v ".:/app" \
  --network todo-app \
  -e MYSQL_HOST=mysql \
  -e MYSQL_USER=root \
  -e MYSQL_PASSWORD=secret \
  -e MYSQL_DB=todos \
  node:24-alpine \
  sh -c "npm install && npm run dev"
```

Copia arquivos fontes de getting-started-app para /app (bind Volume). Se alterar o fontes em getting-started-app, altera também no container.

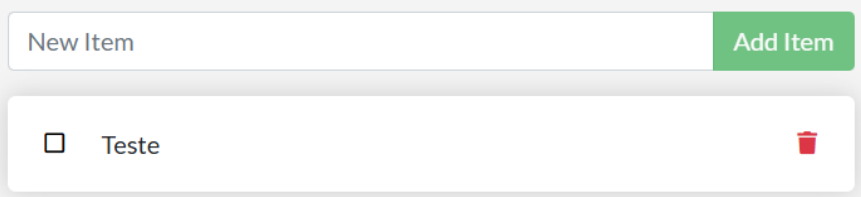
Rede a qual o container será ligado.

Corresponde ao parâmetro "--network-alias mysql" usado na criação do container MySQL.

Usuário, senha e banco de dados.

Imagem

Executa os comandos para instalar dependências (npm install) e ativar o serviço Web.



Testar a aplicação em localhost:3000

Inspecionar o banco de dados

docker exec -it meubd mysql -u root -p

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| todos              |
+--------------------+
5 rows in set (0.02 sec)

mysql> use todos;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+----------------+
| Tables_in_todos |
+----------------+
| todo_items     |
+----------------+
1 row in set (0.00 sec)

mysql> select * from todo_items;
+--------------------------------------+-------+-----------+
| id                                   | name  | completed |
+--------------------------------------+-------+-----------+
| 3a2ece89-1886-4715-8012-de1975e939eb | Teste |         0 |
+--------------------------------------+-------+-----------+
1 row in set (0.00 sec)
```

Compose

Criar arquivo compose.yaml

```yaml
services:
  app:
    image: node:24-alpine
    command: sh -c "npm install && npm run dev"
    ports:
      - 127.0.0.1:3000:3000
    working_dir: /app
    volumes:
      - ./:/app
    environment:
      MYSQL_HOST: mysql
      MYSQL_USER: root
      MYSQL_PASSWORD: secret
      MYSQL_DB: todos

  mysql:
    image: mysql:8.0
    volumes:
      - todo-mysql-data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: todos

volumes:
  todo-mysql-data:
```

Aplicação. Nome na rede. Rede criada automaticamente.

Mapeia o diretório atual da máquina física para o diretório /app do container, copiando todos os arquivos. Instala as dependências e executa a aplicação.

Variáveis de ambiente da aplicação que recebem os dados para conexão com o banco de dados.

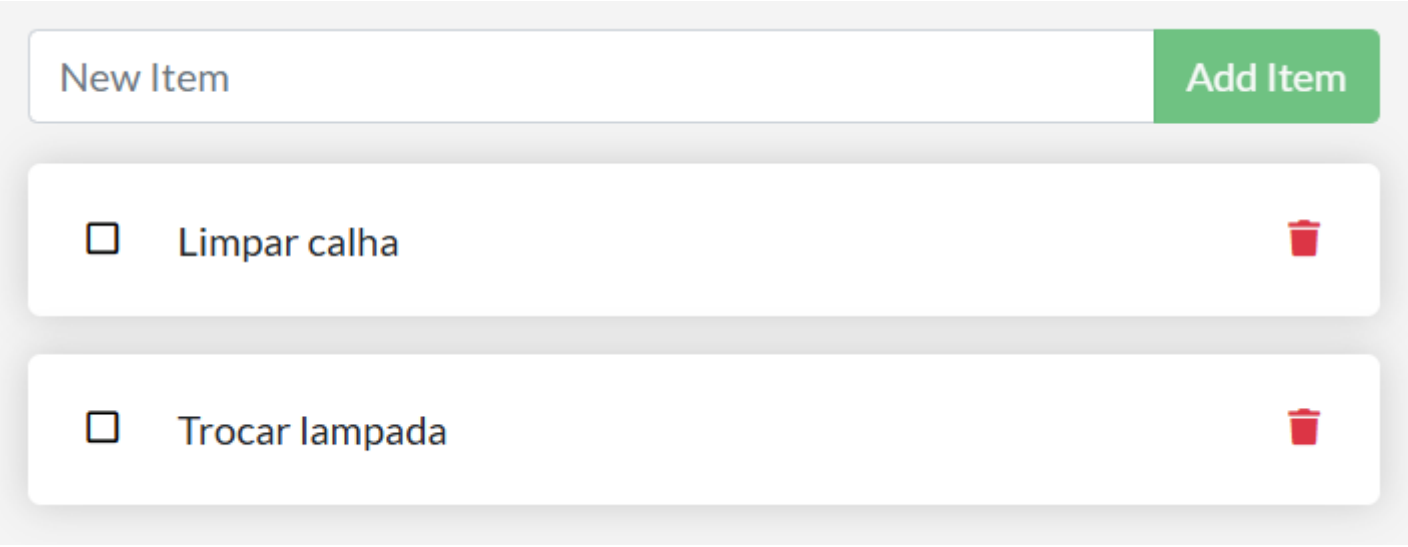
Gerenciador de bancos de dados. Nome na rede.

Endereço no container onde os arquivos do MySQL serão colocados. Este endereço é mapeado para um endereço da máquina física (volume).

Instanciar containers via compose

```
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker volume ls
DRIVER    VOLUME NAME
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
bd898a1beaa4   bridge    bridge    local
7d0af15c4b8f   host      host      local
ff95a0b66170   none      null      local
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker compose up -d
WARN[0000] No services to build
[+] up 4/4
 ✔Network getting-started-app_default        Created
 ✔Volume getting-started-app_todo-mysql-data Created
 ✔Container getting-started-app-app-1        Created
 ✔Container getting-started-app-mysql-1      Created
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker volume ls
DRIVER    VOLUME NAME
local     getting-started-app_todo-mysql-data
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker network ls
NETWORK ID     NAME                          DRIVER    SCOPE
bd898a1beaa4   bridge                        bridge    local
1bd94ffd9311   getting-started-app_default   bridge    local
7d0af15c4b8f   host                          host      local
ff95a0b66170   none                          null      local
prtcandido@PauloAcer:~/testedocker2/getting-started-app$
```

Acessar a aplicação



Modificar a aplicação na máquina física atualiza o código no container

EXPLORER

GETTING-STARTED-APP [WSL: UBUNTU]

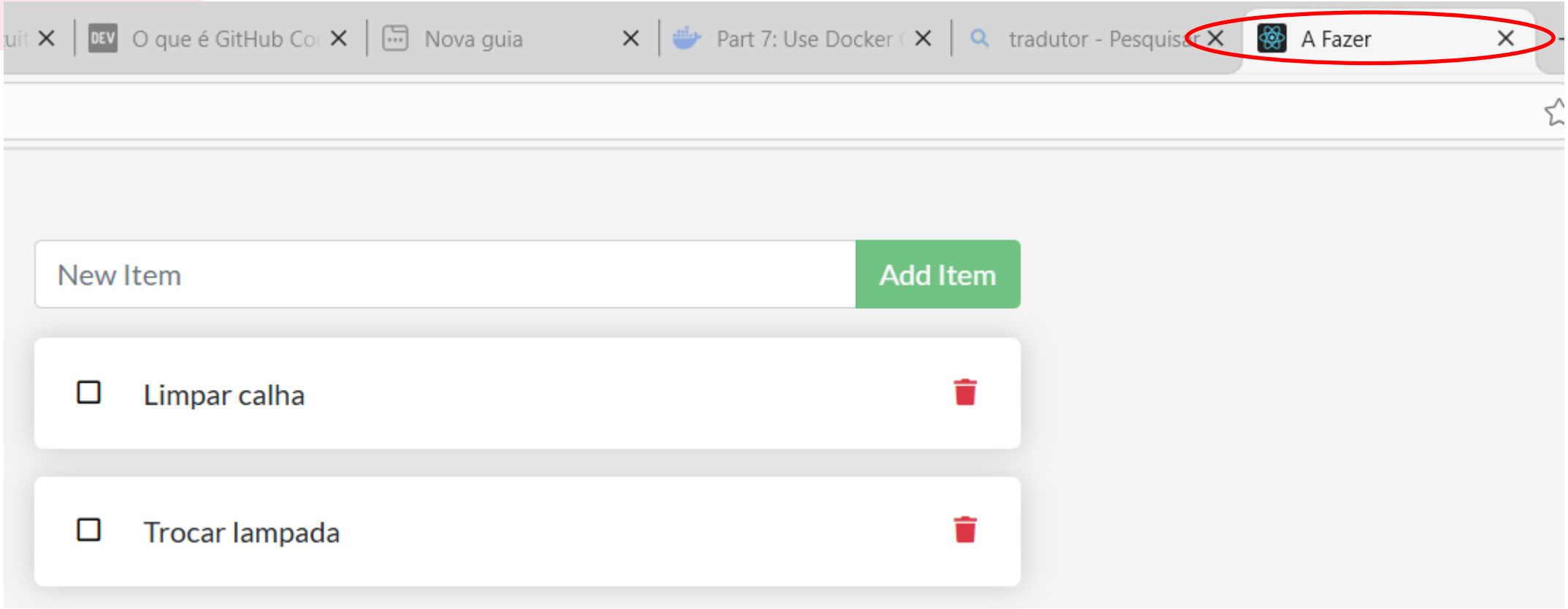
- node_modules
- spec
- src
  - persistence
  - routes
  - static
    - css
    - js
    - index.html
  - index.js
- .dockerignore
- compose.yaml U
- dockerfile U
- package-lock.json
- package.json
- README.md

Untitled-1 | compose.yaml U | index.html

src > static > index.html > html > head > title

```
1
2  <!DOCTYPE html>
3  <html>
4  <head>
5      <meta charset="utf-8" />
6      <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no, maxi
7      <link rel="stylesheet" href="css/bootstrap.min.css" crossorigin="anonymous" />
8      <link rel="stylesheet" href="css/font-awesome/all.min.css" crossorigin="anonymous" />
9      <link href="https://fonts.googleapis.com/css?family=Lato&display=swap" rel="stylesheet" />
10     <link rel="stylesheet" href="css/styles.css" />
11     <title>A Fazer</title>
12 </head>
13 <body>
14     <div id="root"></div>
15     <script src="js/react.production.min.js"></script>
16     <script src="js/react-dom.production.min.js"></script>
17     <script src="js/react-bootstrap.js"></script>
18     <script src="js/babel.min.js"></script>
19     <script type="text/babel" src="js/app.js"></script>
20 </body>
21 </html>
```

Modificar a aplicação na máquina física atualiza o código no container



Remover os container e volume

```
prtcandido@PauloAcer:~/testedocker2/getting-started-app$ docker compose down --volumes
[+] down 4/4
 ✔Container getting-started-app-mysql-1      Removed
 ✔Container getting-started-app-app-1        Removed
 ✔Volume getting-started-app_todo-mysql-data Removed
 ✔Network getting-started-app_default        Removed
prtcandido@PauloAcer:~/testedocker2/getting-started-app$
```